

Fieldpass

Build, Deployment & Internal Test Report

Prepared for: Quad Data Sdn Bhd

Date: 06 May 2026

Version: 1.0

Confidentiality: Internal use

1. Executive Summary

This document summarizes the Fieldpass build workflow, deployment to a production-like server, and internal testing outcomes. It is intended to provide traceability of what was built, where it was deployed, and how it was verified.

Key outcomes

- Fieldpass was successfully deployed to a DigitalOcean Droplet using Nginx and a Docker-based reverse proxy.
- Core user journeys and role-based access were internally tested using a structured checklist.
- No critical blockers found in the covered scenarios; minor UI/validation issues (if any) are tracked separately.

Scope

- Build & packaging overview
- Deployment-to-server report (DigitalOcean droplet + Nginx + Docker reverse proxy)
- Internal testing report (functional + smoke tests)

Out of scope

- Full security penetration testing
- Load/performance benchmarking beyond smoke verification

2. Build & Deployment to Server Report

2.1 Environment

- Cloud provider: DigitalOcean
- Target: Droplet (Linux)
- Web server: Nginx
- Containerization: Docker + Docker Compose
- Reverse proxy: Dockerized reverse proxy fronted by Nginx (TLS termination at Nginx)

2.2 Build artifacts

The deployment package includes the application source, dependency manifests, and container definitions. The build produces a runnable application container image and supporting services (e.g., web server, queue/worker, scheduler) as defined by the Compose configuration.

Build inputs

- Source repository: Fieldpass
- Runtime: PHP 8.2+
- Framework: Laravel (as per project dependency manifest)

2.3 Deployment architecture

High-level request flow:

- Internet → Nginx (Droplet) → Docker reverse proxy → Fieldpass app container(s)
- Static assets are served via the application stack, consistent with environment configuration.

2.4 Deployment steps executed

The following procedure was used to deploy Fieldpass to the droplet:

- Provision droplet and secure access (SSH keys, firewall rules).
- Install Docker Engine and Docker Compose.
- Install and configure Nginx on the droplet.
- Configure Nginx server blocks to forward traffic to the Docker reverse proxy upstream.
- Place environment variables and secrets in the server environment (e.g., *.env*) and restrict file permissions.
- Pull the latest application version (git pull or artifact upload).
- Build and start containers using Docker Compose (*docker compose up -d --build*).
- Run database migrations and cache warms as required (*php artisan migrate, config/cache*).
- Verify health checks: web responds, login page reachable, basic role access verified.

2.5 Verification checklist (deployment)

- Nginx configuration syntax validated and service active.
- Reverse proxy routes to the correct container upstream.
- Application returns HTTP 200 for main routes.
- Admin login reachable; authentication works.

- Logs accessible for troubleshooting (Nginx + container logs).

2.6 Rollback plan

If a deployment fails, rollback is performed by redeploying the previous tagged image/revision and restarting services:

- Switch to previous container image tag or git revision
- *docker compose up -d*
- Verify health checks as above

3. Internal Testing Report

3.1 Testing approach

Internal testing focused on functional smoke coverage of role-based dashboard behavior and primary CRUD workflows. Testing was executed manually using a checklist and validated against expected UI responses and authorization rules.

Test types

- Smoke tests (startup, login, navigation)
- Functional tests (CRUD flows, filters/sorting)
- Authorization checks (role/permission gating)

3.2 Test environment

- Environment: Local development + server deployment verification
- Browser(s): Chromium-based (primary)
- Data: Representative admin + club-linked accounts

3.3 Test coverage (high-level)

Authentication & session

- Admin login with OTP/password flow (as configured)
- Session persistence and logout

Role-based navigation

- Sidebar visibility matches permissions
- Club Manager dashboard renders for club-linked admin

Budget & expenses page

- Page accessible for Club Manager (or users with budget permissions)
- Table grouped by Year and Month
- Filtering by year/month
- Sorting via column headers
- Add expense item with required fields
- Edit expense item via modal
- Delete expense item with confirmation
- Club scoping: items are isolated per club

3.4 Results summary

- Overall status: Pass (for covered scenarios)
- Critical defects: None identified in the tested checklist
- Non-critical issues: Minor issues, if any, tracked separately (UI/UX, copy, edge validation)

3.5 Recommendations

- Add automated feature tests for role-based access and CRUD flows.
- Add basic monitoring/alerting on server: container restarts, HTTP error rate, disk usage.

- Define a periodic backup plan (DB + environment config).

4. Appendix: Deployment & Test Checklists

4.1 Deployment checklist

- DNS points to droplet IP
- TLS certificate installed and auto-renew configured (if applicable)
- Firewall restricts inbound ports (80/443 + SSH)
- Environment variables reviewed and secured
- *docker compose up -d --buildsuccessful*
- Migrations run and verified
- Health checks passed

4.2 Internal test checklist

- Login / logout
- Club dashboard loads for Club Manager
- Budget & expenses visible and accessible
- Add / edit / delete expense flows
- Filter year/month
- Sort columns
- Unauthorized users cannot access budget routes (403)